

TokenOps 3 - Airdrop

Print 

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Airdrop
Timeline	2025-05-13 through 2025-05-15
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	Spec 
Source Code	https://github.com/VestingLabs/tokenops-airdrop-v2  #0d853fc 
Auditors	- Cameron Biniamow Auditing Engineer - Julio Aguilar Auditing Engineer - István Böhm Auditing Engineer

Documentation quality	High 
Test quality	High 
Total Findings	10  Fixed: 6 Acknowledged: 3 Mitigated: 1

High severity findings ⓘ	0
Medium severity findings ⓘ	1 Fixed: 1
Low severity findings ⓘ	4 Fixed: 4
Undetermined severity findings ⓘ	0
Informational findings ⓘ	5 Fixed: 1 Acknowledged: 3 Mitigated: 1

Summary of Findings

The TokenOps Merkle Distributor system enables token distributions using Merkle proofs for verification. It supports both ERC20 and native token (ETH) distributions through dedicated contracts. A factory contract allows users to deploy distribution campaigns with configurable parameters including claim periods and fee structures. The system implements two fee models: gas-based and token-based. Additional features include integration with staking contracts, bonus token functionality, and a grace period that allows the admin to withdraw funds while the airdrop is active but only for 7-day time window after the first claim.

The codebase is well-written, following the best practices and patterns. It also has good documentation and testing. This audit uncovered security issues related to access control, fee handling, and input validation. Notably, the `setPreferredFeeType()` function lacks proper access restrictions, allowing unauthorized users to manipulate fee settings, potentially leading to financial loss. Users may also overpay due to improper fee application during claims, and missing validations on key parameters like the Merkle root and timing windows increase the risk of misconfiguration. We recommend that the team address these concerns before deployment.

Fix-Review Update: The team addressed all issues by fixing or acknowledging them. During the first fix review, we found issues [TOK-3](#), [TOK-4](#), and [TOK-5](#), which were fixed in the following commits.

ID	TOK-1
DESCRIPTION	Missing Access Control Allows Users to Bypass Fees
SEVERITY	• Medium ⓘ

STATUS

Fixed

ID

TOK-2

DESCRIPTION

Users Might Pay More Fees than Expected

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-3

DESCRIPTION

Third-Party Invocation of Claiming Functions May Lead to Unintended Consequences for Beneficiary

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-4

DESCRIPTION

Insufficient Contract Token Balance Can Lead to Fee Collector Not Receiving Full Token Fees

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-5

DESCRIPTION

Disconnected Staking Whitelist Status

SEVERITY

• Low ⓘ

STATUS

Fixed

ID

TOK-6

DESCRIPTION

Staking Bonus Is Not Ensured

SEVERITY

• Informational ⓘ

STATUS

Acknowledged

ID

TOK-7

DESCRIPTION

Potential for Malicious Airdrop Campaigns Due to Permissionless Deployment and User-Controlled Parameters

SEVERITY

• Informational ⓘ

STATUS

Acknowledged

ID

TOK-8

DESCRIPTION

Incomplete Custom Fee Configuration

SEVERITY

• Informational ⓘ

STATUS

Fixed

ID

TOK-9

DESCRIPTION	Staking Gas Fee Cannot Be Updated
SEVERITY	• Informational ⓘ
STATUS	Acknowledged

ID	TOK-10
DESCRIPTION	Missing Input Validation
SEVERITY	• Informational ⓘ
STATUS	Mitigated

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication

- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The scope includes all files in the repo <https://github.com/VestingLabs/tokenops-airdrop-v2> at commit `0d853fcb26485d0563e3b009c02f810d1d060c93`.

Files Included

- contracts/
 - core/BaseTokenOpsMerkleDistributor.sol
 - distributors/
 - TokenOpsMerkleDistributor.sol
 - TokenOpsMerkleDistributorNative.sol
 - factory/
 - TokenOpsMerkleDistributorFactory.sol
 - interfaces/
 - ITokenOpsMerkleDistributor.sol
 - ITokenOpsMerkleDistributorBase.sol
 - ITokenOpsMerkleDistributorFactory.sol
 - ITokenOpsMerkleDistributorNative.sol
 - ITokenOpsStaking.sol
 - ITypes.sol

Operational Considerations

1. Users can `claimAndStake()` in the `TokenOpsMarketDistributor` contract, which grants the user bonus reward tokens from the reward owner on a first-come, first-served basis until the bonus amount is exhausted. The reward owner must also grant the required allowances to the `TokenOpsMarketDistributor` contract. If the reward owner has an insufficient balance of reward tokens or the allowance is exhausted, no bonus will be granted to the user, and only the claimed amount of tokens will be staked.
2. Reward token claims are only available to users within the claim window. Users will not be able to claim reward tokens before the claim window begins or after it expires. Further, users can only claim if the distributor contract has a sufficient balance. There are no guarantees that the distributor contract will have a sufficient reward token balance to cover all claims.
3. The distributor contracts' `owner` can withdraw unclaimed reward tokens at any time except when the claim window is active and the grace period (seven days after the first claim is executed) has expired.
4. The creator of a `TokenOpsMarketDistributor` contract will set the address of the corresponding staking contract. However, there are no guarantees that the staking contract is valid or that it can accept stakes of the distributed reward token. Users should always verify that the staking contract is a valid TokenOps staking contract that allows staking of the distributed reward token. Additionally, the staking contracts may also incur additional fees, and users' tokens may be temporarily locked based on the rules of the staking contract.
5. The `TokenOpsMarketDistributor` contract inherits `ReentrancyGuardTransient`, which is incompatible with some chains. Ensure the contract is deployed on a chain supporting the `TSTORE` and `TLOAD` opcodes.
6. The codebase does not support ERC20 tokens with fee-on-transfer or rebase mechanisms. Campaign creators should carefully select compatible tokens to avoid unexpected behavior.
7. Private keys associated with privileged roles should be securely managed and, depending on their level of risk and responsibility, should be operated using a multisig or cold wallet.

Key Actors And Their Capabilities

`TokenOpsMerkleDistributor`

Fee Collector

- Can transfer the Fee Collector role to any other address.
- Can withdraw accrued reward token fees from the distributor contract.
- Can withdraw accrued gas fees from the distributor contract.

Owner

- Can withdraw unclaimed reward tokens from the distributor contract at any time except when the claim window is active and the grace period (seven days after the first claim is executed) has expired.
- Can withdraw any amount of non-reward tokens from the distributor contract.
- Can transfer ownership of the distributor contract to any other address.

Reward Owner

- Holds excess reward tokens used as bonus tokens when users claim and stake.

Users

- Can claim reward tokens with a valid Merkle proof when the claim window is active.

TokenOpsMerkleDistributorNative

Fee Collector

- Can transfer the Fee Collector role to any other address.
- Can withdraw accrued gas fees from the distributor contract.

Owner

- Can withdraw unclaimed reward tokens from the distributor contract at any time except when the claim window is active and the grace period (seven days after the first claim is executed) has expired.
- Can transfer ownership of the distributor contract to any other address.

Users

- Can claim reward tokens with a valid Merkle proof when the claim window is active.

TokenOpsMerkleDistributorFactory

Owner

- Can enable custom fee types and fee rates for any campaign creator.
- Can disable custom fee types and fee rates for any campaign creator.
- Can update any campaign creator's preferred fee type and rate.
- Can reset the default gas fee to zero.
- Can reset the default token fee to zero.
- Can update the fee collector's address.
- Can update the default gas fee.
- Can update the default token fee.
- Can update the default fee type.
- Can transfer ownership of the factory contract to any other address.

Campaign Creators

- Can create a new Merkle distributor contract with the following customizable parameters:
 - Token to distribute.
 - Merkle root for validating claims.
 - Start and end time of the claim window.
 - Stake contract where users can optionally stake their claimed tokens.
 - Reward owner who holds bonus tokens that are given out to users who claim and then stake their tokens
 - Bonus percentage.
- Can create a new native Merkle distributor contract with the following customizable

parameters:

- Merkle root for validating claims.
- Start and end time of the claim window.

Findings

TOK-1

Missing Access Control Allows Users to Bypass Fees

• Medium ⓘ

Fixed



Update

Marked as "Fixed" by the client.

Addressed in: `70df9ec462d46988a63faf402ae481dc04f4f80f` .

The client provided the following explanation:

`We've made the preferred fee setup owner only.`



Update

The client changed the approach to use role-based access control instead of the owner in commit `2c06b09e5af4f3ad621af0add00db169601b670` .



Update

The function `setPreferredFeeType()` was later removed in `a0082c3855bad17bca37cbf04ad40e99f64c4951`.

File(s) affected: `contracts/factory/TokenOpsMerkleDistributorFactory.sol`

Description: The `setPreferredFeeType()` function in the `TokenOpsMerkleDistributorFactory` contract allows any caller to change the preferred fee type for any campaign creator. This opens the door to frontrunning attacks, where an attacker could modify the fee type just before a new `TokenOpsMerkleDistributor` is deployed. If the selected fee type has no corresponding value set, the resulting fee could unintentionally be zero. Since the fee type and value are stored as immutable variables in the distributor contract, this would require a re-deployment to correct and result in a loss of fees for the fee collector.

Exploit Scenario:

1. Alice configures `DistributionToken` as the preferred fee type with a 10% fee, leaving `gasFee` equal to zero.

2. Bob calls `setPreferredFeeType()` with Alice as `campaignCreator` and sets their preferred fee type from `DistributionToken` to `Gas`.
3. Alice does not realize the change (or gets front-ran by Bob) and deploys the airdrop contract with the configured 0% Gas fee.
4. Alice cannot change the fee configuration of the deployed airdrop contract, so they need to potentially claw back any transferred tokens and redeploy the airdrop contract.

Recommendation: We recommend allowing only the owner or the campaign creator to have access to this function.

TOK-2 Users Might Pay More Fees than Expected

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `b7bfd5eb50827e85e20b8ed820cef2e74499dbc7`.

The client provided the following explanation:

We now calculate fees based on whether the action is a claim or claimAndStake. Additionally, we identified an inconsistency in the staking amount when the feeType is set to "distribution token fee" – the fee was being deducted from the airdrop amount, but the full airdrop amount was still being sent to the staking contract. We've addressed this by adjusting the stakingAmount to be `airdropAmount - feeAmount` to ensure consistency.

For better cohesion with the staking contract, we've also added the whitelist behavior.

i Update

The whitelist behavior mentioned above was removed in commit `cf6d331a8ae8257ebeec65ba56c69b419eaaa76c`.

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The `TokenOpsMerkleDistributor` contract offers two claim methods: one for claiming tokens and another for claiming and staking. While users who stake are expected to pay an additional `STAKING_GAS_FEE` on top of the regular claim fee, the internal `_claim()` function does not differentiate between the two paths and always charges the combined amount. As a result, users who only intend to claim may overpay.

Recommendation: Modify the logic to charge only the base claim fee for standard claims, and

apply `fee + STAKING_GAS_FEE` when the user opts to stake as part of the claim. Make sure the function differentiates between standard and staking claims and the different fee types between the claiming and staking functionalities.

TOK-3

Third-Party Invocation of Claiming Functions May Lead to Unintended Consequences for Beneficiary

• Low ⓘ Fixed

✓ Update

This was resolved in commit `2c06b09e5af4f3ad621af0add00db169601b670` by adding the `onlyAuthorizedClaimer()` modifier to the `claim()` and `claimAndStake()` functions. The modifier ensures that the caller is either the account itself or holds the admin or claimer role, provided the staking contract address has been initialized.

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The `claim()` and `claimAndStake()` functions allow any address to initiate a claim for a beneficiary provided they provide a valid Merkle proof for that account. This design enables scenarios where:

- If a third party calls `claimAndStake()`, the account's tokens may be locked without their consent.
- If a third party calls `claim()`, the account is excluded from the bonus tokens received when calling the `claimAndStake()` function.

Recommendation: Consider the option to restrict claims such that `msg.sender == account`.

TOK-4

Insufficient Contract Token Balance Can Lead to Fee Collector Not Receiving Full Token Fees

• Low ⓘ Fixed

✓ Update

This was resolved as recommended in commit `2c06b09e5af4f3ad621af0add00db169601b670`.

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributor.sol`,

contracts/distributors/TokenOpsMerkleDistributorNative.sol

Description: In the `TokenOpsMerkleDistributor` contract, when `feeType` is `DistributionToken`, the `_claim()` function calculates a `feeAmount` from the user's gross claim amount and adds this `feeAmount` to the `numTokenReservedForFee` state variable, then it transfers the `transferAmount` (gross amount minus `feeAmount`) to the user. The contract implicitly assumes that it holds enough tokens to cover all potential user claims plus the fees that will be accrued in `numTokenReservedForFee`.

However, the `_claim()` function does not verify that the token balance of the contract is sufficient to satisfy both the current `transferAmount` to the user and the existing `numTokenReservedForFee`. If the contract is underfunded, user claims will be processed as long as the contract has enough tokens. This can deplete the contract's token balance to a level below `numTokenReservedForFee`. Consequently, when the `feeCollector` attempts to call `withdrawTokenFee()`, the call may fail because of the token balance and `numTokenReservedForFee` inconsistency.

The `TokenOpsMerkleDistributorNative` contract verifies that its native token balance is higher than the claimed amount and the accrued fee (`numNativeReservedForFee`), otherwise it reverts with the `NotEnoughNativeTokens` error. However, it incorrectly compares the token balance with `numNativeReservedForFee` before increasing it with the fees for the current claim. This can also lead to cases where there are fewer native tokens left in the contract than what is accounted for in the `numTokenReservedForFee` variable.

Recommendation: Consider checking that the contracts have enough tokens to cover both the claimed amount and the accrued fees.

TOK-5 Disconnected Staking Whitelist Status

• Low ⓘ Fixed

✓ Update

The staking whitelist feature was removed from the airdrop contracts in commit `cf6d331a8ae8257ebeec65ba56c69b419eaaa76c`.

Description: During contract construction, the status of the whitelist is fetched from the `STAKING` contract and set in the immutable state variable `STAKING_WHITELIST_ENABLED`. However, since the whitelist status is mutable in the `STAKING` contract, the whitelist status may become out of sync between the two contracts. Therefore, if the whitelist was disabled at the time of deployment and later enabled, all `claimAndStake()` calls will fail since the `STAKING.stake()` call expects the whitelist to be disabled. Alternatively, if the whitelist was enabled at the time of deployment and then later disabled, some users may be unable to `claimAndStake()` if the user is not in the `STAKING` contract's Merkle tree.

Recommendation: In the `claimAndStake()` function, fetch the whitelist status directly from the `STAKING` contract rather than caching the status upon deployment.

TOK-6 Staking Bonus Is Not Ensured

• **Informational** ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

The reward bonus is first come first serve and we don't want to make it mandatory for that. Our UI/Any UI should track the reward amount available for the recipients claim.

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The bonus tokens are transferred from the `REWARD_OWNER` address to the `TokenOpsMerkleDistributor` contract during the `claimAndStake()` call. If this transfer fails (e.g., `REWARD_OWNER` has not approved enough tokens to this distributor contract, or `REWARD_OWNER`'s balance is insufficient), `trySafeTransferFrom` returns false. However, instead of reverting, the code handles this by not adding the bonus to the staking amount. Depending on the staking rules, this could have adverse consequences for the users, as they could simply claim their tokens.

It is also noted that the campaign creator sets the `STAKING` and `REWARD_OWNER` addresses, and everybody can create airdrop campaigns using the factory. The campaign creator can just promise a huge bonus for staking, then lock the user's tokens in the staking contract without paying out the bonus tokens, while the user pays the claiming fees.

Recommendation: Consider reverting if the bonus transfer fails. Otherwise, consider indicating in the documentation the associated risks and that the bonus may only be available in a limited amount.

TOK-7

Potential for Malicious Airdrop Campaigns Due to Permissionless Deployment and User-Controlled Parameters

• **Informational** ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

Our contracts are designed with a strong focus on permissionlessness, so we aim to minimize restrictions on functionality wherever possible. Through our UI, we also offer integration with Sablier, which is itself permissionless and supports any token. Additionally, our UI may notify users about third-party staking contracts beyond our own.

File(s) affected: `contracts/interfaces/ITokenOpsMerkleDistributorFactory.sol` , `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The factory contract allows any account (`campaignCreator`) to deploy new ERC20 airdrops. During deployment, the campaign creator can control the `tokenAddress` (for ERC20 airdrops), the `staking` contract address (for `claimAndStake` functionality), and the `rewardOwner` address (for paying out staking bonuses). This permissionless design, while flexible, creates an avenue for malicious actors to launch deceptive airdrop campaigns. For example:

- Fake or malicious `tokenAddress` : The token address could point to a worthless token, one with deceptive properties, or one designed to exploit users who interact with it.
- Malicious `staking` contracts: The staking address could point to a contract controlled by the attacker. Users opting for the `claimAndStake` feature, potentially lured by high `bonusPercentage` or limited offers, would have their claimed tokens transferred to this malicious staking contract. The staking contract could include backdoors for the attacker to steal all deposited tokens or fail to deliver any promised bonuses.
- Unfunded `rewardOwner` addresses: The rewardOwner for staking bonuses might not possess or approve the necessary tokens, leading to users staking their assets without receiving the advertised bonus, even if the staking contract itself isn't directly malicious.

Victims could lose any tokens sent to malicious staking contracts and any fees paid during the claim process. The reputation of any platform or UI utilizing this factory could also be damaged if users fall prey to such schemes. While the factory contract operates as coded, the system it enables can be misused to harm users.

Recommendation: Consider only allowing the use of verified tokens and staking contracts.

Otherwise, consider displaying warnings for the claimers in the UI/Frontend and in the documentation about the potential risks, and clearly communicate that airdrops are user-generated and not vetted by the platform. Also, consider designing the UI/frontend to help users discover potential malicious airdrops or staking contracts, e.g., by showing users the full addresses for the associated contracts.

TOK-8 Incomplete Custom Fee Configuration

• Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `aaae380ab0496b0123a9686d3a834d026ae2495f` .

The client provided the following explanation:

```
We now revert on preferred fee setup if custom fee isn't already set.
```

i Update

The `setPreferredFeeType()` function was removed from the contract in commit `a0082c3855bad17bca37cbf04ad40e99f64c4951` , and now `setCustomFee()` configures the preferred fee type.

File(s) affected: `contracts/factory/TokenOpsMerkleDistributorFactory.sol`

Description: The `setPreferredFeeType()` function in the `TokenOpsMerkleDistributorFactory` contract enables a custom fee configuration without verifying that the actual fee values (`gasFee` or `tokenFee`) were previously set. This could expose users to making incomplete fee configurations by mistake.

Recommendation: We recommend reverting if `customFee.enabled` is false to make sure it was already configured before.

TOK-9 Staking Gas Fee Cannot Be Updated

• Informational ⓘ

Acknowledged

i Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

```
STAKING_GAS_FEE is derived from an immutable variable in the staking contract, ensuring that the fee amount remains constant and cannot be changed.
```

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The `STAKING_GAS_FEE` variable is immutable. It will not be possible to update this variable if the staking contract updates its fee during the airdrop, preventing the users from calling the `claimAndStake()` function.

Recommendation: Consider allowing the users to refresh the cached fee value.

TOK-10 Missing Input Validation

• Informational ⓘ

Mitigated

✓ Update

Marked as "Fixed" by the client.

Addressed in: `f7cf4a6c70effa5623795ffbb4fff7b3ad41c3a5`.

The client provided the following explanation:

We've added the missing validations to the contracts.

`defaultGasFee`: While we're not enforcing limits at the contract level, the UI will include warnings and indicators to guide users appropriately.

Long claim window: This will also be addressed through UI warnings to inform users of any potential concerns.

File(s) affected: `contracts/factory/TokenOpsMerkleDistributorFactory.sol`,
`contracts/core/BaseTokenOpsMerkleDistributor.sol`,
`contracts/distributors/TokenOpsMerkleDistributor.sol`

Related Issue(s): [SWC-123](#)

Description: It is important to validate inputs, even if they only come from trusted addresses, to avoid human error. The following is a non-exhaustive list of missing input validations that could create issues such as users not being able to claim:

1. The `defaultGasFee` and `defaultTokenFee` state variables in the factory constructor can be initialized with any value up to `uint256.max()`. To avoid misconfiguration or accidental overcharging, upper bounds should be enforced, especially for `defaultTokenFee`, which represents a percentage and should not exceed `BASIS_POINTS`, otherwise, users won't be able to claim. The same validation should be applied in the `setDefaultGasFee()`, `setDefaultTokenFee()`, and `setCustomFee()` functions.
2. The constructor of the `BaseTokenOpsMerkleDistributor` does not validate the `merkleRoot_` parameter. Since it is immutable, it should be checked against `bytes32(0)` to ensure valid claims can be proven. Otherwise, users will be unable to validate their eligibility.
3. The `BaseTokenOpsMerkleDistributor` constructor does not enforce constraints on the

claim window (`endTime_ - startTime_`). If the window is set excessively long (e.g., 10 years) and the grace period has passed, admins may be unable to withdraw unclaimed funds in a timely manner.

4. The `TokenOpsMerkleDistributor` constructor does not verify that `fee_ <= BASIS_POINTS` when `feeType_` is `ITypes.FeeType.DistributionToken` . If not checked, users may be blocked from claiming.
5. The `setPreferredFeeType()` function does not validate that the provided `preferredFeeType` matches either the `FeeType.Gas` or `FeeType.DistributionToken` . This could result in selecting an unsupported fee type.
6. The functions `setCustomFee()` , `setPreferredFeeType()` , `disableCustomFee()` , and `enableCustomFee()` accept `campaignCreator` as an input but do not validate that the address is non-zero, which could lead to unintended behavior or incorrect fee configurations.
7. In the `TokenOpsMerkleDistributor._claim()` function, when `feeType_` is `DistributionToken` , there is no check to ensure that `msg.value` is either `0` (for standard claims) or equal to `STAKING_GAS_FEE` when staking is included. Without this validation, users may accidentally send unnecessary native tokens, leading to a loss of funds.
8. In the `_claim()` function, the `if (amount == 0) revert ZeroBalance();` check is only enforced when `feeType == ITypes.FeeType.DistributionToken` . To improve consistency and avoid unnecessary processing, consider moving this check to the beginning of all claim functions, regardless of the fee type.

Recommendation: We recommend adding the relevant checks.

Auditor Suggestions

S1

Inconsistent and Unmet Admin Withdrawal Requirements

Fixed

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We've updated the doc and it's the following behavior we expect.

Grace period to clawback before `endTime`:

- If before `startTime`, clawback IS possible
- If `startTime` has passed but no claims made, clawback IS possible
- If claims made but 7 days haven't passed from the first

- claim, clawback IS possible
- If claims made and 7 days have passed, clawback IS NOT possible (unless expired)

File(s) affected: `contracts/distributors/TokenOpsMerkleDistributorNative.sol`, `contracts/distributors/TokenOpsMerkleDistributor.sol`

Description: The documented specification for the airdrop contract outlines clawback behavior (see below) that appears internally inconsistent. Specifically, it states that clawback should **not** be possible after `startTime` if no claims have been made, yet clawback **is** allowed before `startTime`, and again if claims have been made but less than 7 days have passed. This implies that the admin is granted more control once claims are made, which is unintuitive.

Additionally, the current implementation of `withdraw()` does **not** enforce the second condition. If no claims have been made (i.e., `_firstClaimTime == 0`), the `_hasGracePeriodPassed()` function returns `false`, allowing clawback regardless of whether `startTime` has passed. While this diverges from the written spec, it results in a more coherent and flexible design that favors admin control and avoids the inconsistency.

- Grace period **to** clawback **before** `endTime`
 - If **before** `startTime`, clawback IS possible
 - If `startTime` has passed **but** no claims made, clawback IS NOT possible (unless expired)
 - If claims made **but** 7 days haven't passed **from the first** claim, clawback IS possible
 - If claims made **and** 7 days have passed, clawback IS NOT possible (unless expired)

Recommendation: We recommend either updating the specification to align with the current implementation or revisiting the intended behavior to ensure clarity.

S2 General Recommendations

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `48f22b0b60aea236791fc0aeb20f4b42ba00ee1d`.

The client provided the following explanation:

Fix the issues as suggested.

Description: The following is a non-exhaustive list of possible improvements:

1. The function `newMerkleDistributor()` calls `_getFeeType()` and `_getFee()` twice. Consider calling it once and using the value twice to be more efficient.
2. The constructor of the `TokenOpsMerkleDistributor` contract has a try-catch block to set the `STAKING_GAS_FEE` variable. The return variable from the try call is called `feeType`, which is shadowing the state variable from the `BaseTokenOpsMerkleDistributor` contract. Consider renaming the returned variable to `stakingFeeType`.

Recommendation: Consider applying the changes mentioned above.

S3

Critical Role Transfer Not Following Two-Step Pattern

Acknowledged

Update

Marked as "Acknowledged" by the client.

The client provided the following explanation:

It's highly unlikely to change and is intended for internal use only, reserved for rare scenarios where a role transfer might be necessary. We prefer to avoid adding complexity for such edge cases.

File(s) affected: `contracts/factory/TokenOpsMerkleDistributorFactory.sol`,
`contracts/core/BaseTokenOpsMerkleDistributor.sol`

Description: The owner of the factory contract can call `transferOwnership()` to transfer the ownership to a new address. If an uncontrollable address is accidentally provided as the new owner address then the contract will no longer have an active owner, and functions with the `onlyOwner` modifier can no longer be executed.

Recommendation: Consider using OpenZeppelin's `Ownable2Step` contract to adopt a two-step ownership pattern in which the new owner must accept their position before the transfer is complete.

S4 Inconsistent Fee Types

Fixed

✓ Update

Marked as "Fixed" by the client.

Addressed in: `496d3a263523cc5be9eb35572dd3266beb419e5f` .

The client provided the following explanation:

`Fixed as suggested.`

i Update

The function was redesigned in commit

`a0082c3855bad17bca37cbf04ad40e99f64c4951` , and uses the correct type.

File(s) affected: `contracts/factory/TokenOpsMerkleDistributorFactory.sol`

Description: The `setCustomFee()` function takes `newFee` as `uint240` , but the `CustomFee.gasFee` and `CustomFee.tokenFee` struct members are defined as `uint256` . While `uint240` fits into `uint256` , this inconsistency might be unintentional.

Recommendation: Consider using `uint256` for the `newFee` parameter to ensure consistency.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means,

such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The test suite contained 142 test cases. All tests were run successfully with the following commands:

```
> pnpm install
> forge test
```

Fix-Review Update: The client added additional tests, and the test suite now includes 184 test cases.

```
Warning: optimizer settings have been disabled for accurate coverage
reports, if you encounter "stack too deep" errors, consider using `
-ir-minimum` which enables viaIR with minimum optimization resolving
most of the errors
```

```
[.] Compiling...
```

```
[..] Compiling 62 files with Solc 0.8.29
```

```
[.] Solc 0.8.29 finished in 3.44s
```

Compiler run successful with warnings:

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.

```
--> test/TokenOpsMerkleDistributor.t.sol:82:17:
```

```
|
82 |     function stake(uint256 _amount, address _staker) external
payable override {
|                                     ^^^^^^^^^^^^^^^^^^^^^
```

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.

```
--> test/TokenOpsMerkleDistributor.t.sol:82:34:
```

```
|
82 |     function stake(uint256 _amount, address _staker) external
payable override {
|                                     ^^^^^^^^^^^^^^^^^^^^^
```

Warning (2072): Unused local variable.

```
--> test/TokenOpsMerkleDistributor.t.sol:704:3:
```

```
|
704 |         bytes32 feeCollectorRole =
distributor.FEE_COLLECTOR_ROLE();
```

```

|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:738:3:
|
738 |         bytes32 feeCollectorRole =
distributor.FEE_COLLECTOR_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:797:3:
|
797 |         bytes32 defaultAdminRole =
distributor.DEFAULT_ADMIN_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:799:3:
|
799 |         bytes32 claimerRole = distributor.CLAIMER_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:881:3:
|
881 |         bytes32 defaultAdminRole =
distributor.DEFAULT_ADMIN_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:883:3:
|
883 |         bytes32 claimerRole = distributor.CLAIMER_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:899:3:
|
899 |         bytes32 defaultAdminRole =
distributor.DEFAULT_ADMIN_ROLE();
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributor.t.sol:901:3:
|
901 |         bytes32 claimerRole = distributor.CLAIMER_ROLE();

```

```
|
|      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Warning (2072): Unused local variable.
--> test/TokenOpsMerkleDistributorNative.t.sol:1106:3:
|
1106 |         uint256 contractBalance =
address(distributor).balance;
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Analysing contracts...
Running tests...
test/TokenOpsMerkleDistributor.t.sol:TokenOpsMerkleDistributorTest
↳ Suite result: ok. 97 passed; 0 failed; 0 skipped; finished in
165.75ms (139.19ms CPU time)
test/TokenOpsMerkleDistributorNative.t.sol:TokenOpsMerkleDistributor
NativeTest
↳ Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in
165.49ms (242.05ms CPU time)
test/TokenOpsMerkleDistributorFactory.t.sol:TokenOpsMerkleDistributo
rFactoryTest
↳ Suite result: ok. 43 passed; 0 failed; 0 skipped; finished in
165.52ms (130.23ms CPU time)

Ran 97 tests for
test/TokenOpsMerkleDistributor.t.sol:TokenOpsMerkleDistributorTest
[PASS] test_ClaimAndStake_BasicFunctionality() (gas: 3282239)
[PASS] test_ClaimAndStake_CorrectlyCalculatesBonus() (gas: 9626053)
[PASS] test_ClaimAndStake_GasFeeType_HandlesZeroFee() (gas: 3210829)
[PASS] test_ClaimAndStake_RecordsFirstClaimTimeCorrectly() (gas:
3265934)
[PASS] test_ClaimAndStake_ReentrancyProtection() (gas: 3705580)
[PASS] test_ClaimAndStake_RevertIfStakingContractNotSet() (gas:
3120440)
[PASS] test_ClaimAndStake_RevertsWhenClaimNotStarted() (gas:
3121086)
[PASS] test_ClaimAndStake_RevertsWhenClaimWindowHasEnded() (gas:
3123370)
[PASS] test_ClaimAndStake_RevertsWhenIndexAlreadyClaimed() (gas:
3229330)
[PASS] test_ClaimAndStake_RevertsWithInvalidMerkleProof() (gas:
3128690)
[PASS] test_ClaimAndStake_TokenFeeType_DeductsFeeCorrectly() (gas:
3290809)
[PASS] test_ClaimAndStake_WithoutAvailableBonus() (gas: 3219201)
[PASS] test_Claim_AllowsClaimsWithValidMerkleProof() (gas: 3218468)
[PASS] test_Claim_GasFeeType_AcceptsClaimWhenMsgValueEqualsFee()
```

```
(gas: 3212156)
[PASS] test_Claim_GasFeeType_RevertsWhenMsgValueLessThanFee() (gas:
3129490)
[PASS] test_Claim_GasFeeType_RevertshenMsgValueGreaterThanFee()
(gas: 3129459)
[PASS] test_Claim_RecordsFirstClaimTimeCorrectly() (gas: 3274022)
[PASS] test_Claim_RevertWhenNotEnoughToken() (gas: 3159401)
[PASS] test_Claim_RevertsWhenClaimNotStarted() (gas: 3127852)
[PASS] test_Claim_RevertsWhenClaimWindowHasEnded() (gas: 3130137)
[PASS] test_Claim_RevertsWhenIndexAlreadyClaimed() (gas: 3221642)
[PASS] test_Claim_RevertsWithInvalidMerkleProof() (gas: 3133596)
[PASS] test_Claim-TokenFeeType_RevertsWhenAmountIsZero() (gas:
3119653)
[PASS] test_Claim-TokenFeeType_TracksNumTokenReservedForFee() (gas:
3278963)
[PASS]
test_Claim-TokenFeeType_TransfersCorrectAmountAfterFeeDeduction()
(gas: 3226916)
[PASS] test_Claim_WithZeroGasFee() (gas: 3208916)
[PASS] test_Claim_WithZeroTokenFee() (gas: 3209862)
[PASS] test_Constructor_FallbackStakingGasFee() (gas: 3038380)
[PASS] test_Constructor_InitializesCorrectly() (gas: 3121974)
[PASS] test_Constructor_RevertsWhenAdminIsZeroAddress() (gas: 60252)
[PASS] test_Constructor_RevertsWhenBonusPercentageIsInvalid() (gas:
267011)
[PASS] test_Constructor_RevertsWhenEndTimeIsInPast() (gas: 59785)
[PASS] test_Constructor_RevertsWhenFeeCollectorIsZeroAddress() (gas:
57889)
[PASS] test_Constructor_RevertsWhenFeeIsTooHigh() (gas: 273372)
[PASS] test_Constructor_RevertsWhenRewardOwnerIsInvalid() (gas:
266825)
[PASS] test_Constructor_RevertsWhenStartTimeAfterEndTime() (gas:
60072)
[PASS] test_Constructor_RevertsWhenStartTimeEqualEndTime() (gas:
60035)
[PASS] test_Constructor_RevertsWhenTokenAddressIsZero() (gas:
266280)
[PASS]
test_GracePeriodRemainingTime_ReturnsCorrectValueDuringGracePeriod()
(gas: 3211879)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroAfterGracePeriod()
(gas: 3211667)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroBeforeFirstClaim()
(gas: 3105956)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroBeforeStartTime()
(gas: 3104724)
```

```
[PASS] test_GracePeriodStatus_ReturnsFalseAfterStartTime() (gas: 3106807)
[PASS]
test_GracePeriodStatus_ReturnsFalseAfterStartTimeBeforeFirstClaim()
(gas: 3109508)
[PASS] test_GracePeriodStatus_ReturnsFalseOnStartTime() (gas: 3106630)
[PASS] test_GracePeriodStatus_ReturnsTrueAfterGracePeriod() (gas: 3211599)
[PASS] test_OnlyAuthorizedClaimer_AdminRoleTransfer() (gas: 3249169)
[PASS] test_OnlyAuthorizedClaimer_AllowsAdminClaimingForOthers()
(gas: 3204403)
[PASS]
test_OnlyAuthorizedClaimer_AllowsClaimerRoleClaimingForOthers()
(gas: 3305736)
[PASS] test_OnlyAuthorizedClaimer_AllowsSelfClaimingWithStaking()
(gas: 3203878)
[PASS] test_OnlyAuthorizedClaimer_AllowsWhenNoStaking() (gas: 3197374)
[PASS]
test_OnlyAuthorizedClaimer_ClaimAndStake_AllowsAdminClaimingForOthers()
(gas: 3213413)
[PASS]
test_OnlyAuthorizedClaimer_ClaimAndStake_AllowsClaimerRoleClaimingForOthers()
(gas: 3314745)
[PASS]
test_OnlyAuthorizedClaimer_ClaimAndStake_AllowsSelfClaimingWithStaking()
(gas: 3212844)
[PASS]
test_OnlyAuthorizedClaimer_ClaimAndStake_RevertsUnauthorizedClaim()
(gas: 3134797)
[PASS] test_OnlyAuthorizedClaimer_EdgeCase_ZeroAddressAccount()
(gas: 3111528)
[PASS] test_OnlyAuthorizedClaimer_MultipleRolesInteraction() (gas: 3437083)
[PASS] test_OnlyAuthorizedClaimer_RevertsUnauthorizedClaim() (gas: 3134221)
[PASS] test_OnlyAuthorizedClaimer_RoleRevocation() (gas: 3162090)
[PASS] test_OnlyAuthorizedClaimer_StakingContractChange() (gas: 3199420)
[PASS] test_Roles_AdminCanGrantAnyRole() (gas: 3302735)
[PASS] test_Roles_AdminRoleTransfer() (gas: 3248810)
[PASS] test_Roles_BadRenounceConfirmationFails() (gas: 3109654)
[PASS] test_Roles_CorrectRoleHierarchy() (gas: 3117515)
[PASS] test_Roles_EnumerationWorks() (gas: 3355138)
[PASS] test_Roles_FeeCollectorCanGrantFeeCollectorRole() (gas:
```

```
3190573)
[PASS] test_Roles_FeeCollectorCanRenounceRole() (gas: 3035731)
[PASS] test_Roles_FeeCollectorCanRevokeFeeCollectorRole() (gas:
3130917)
[PASS] test_Roles_FeeCollectorCannotGrantOtherRoles() (gas: 3123936)
[PASS] test_Roles_GrantRoleToZeroAddressFails() (gas: 3113686)
[PASS] test_Roles_InitialSetup() (gas: 3126660)
[PASS] test_Roles_RegularUserCannotGrantRoles() (gas: 3130517)
[PASS] test_UpdateFeeCollector_ChainOfUpdates() (gas: 3339602)
[PASS] test_UpdateFeeCollector_NewFeeCollectorCanWithdrawFees()
(gas: 3324917)
[PASS]
test_UpdateFeeCollector_OldFeeCollectorCannotWithdrawAfterUpdate()
(gas: 3240896)
[PASS] test_UpdateFeeCollector_OnlyCallableByFeeCollector() (gas:
3185970)
[PASS] test_UpdateFeeCollector_RevertsForZeroAddress() (gas:
3106173)
[PASS] test_UpdateFeeCollector_RevertsWhenCalledByNonFeeCollector()
(gas: 3120054)
[PASS] test_WithdrawGasFee_AllowsFeeCollectorToWithdrawGasFees()
(gas: 3251897)
[PASS] test_WithdrawGasFee_AllowsPartialWithdrawal() (gas: 3264572)
[PASS] test_WithdrawGasFee_HandlesFailedTransfers() (gas: 3225776)
[PASS] test_WithdrawGasFee_RevertWhenNotTheFeeCollector() (gas:
3216903)
[PASS] test_WithdrawGasFee_RevertWithAmountGreaterThanBalance()
(gas: 3212963)
[PASS] test_WithdrawGasFee_RevertsForZeroAddressRecipient() (gas:
3211884)
[PASS] test_WithdrawGasFee_RevertsIfContractBalanceIsZero() (gas:
3105463)
[PASS] test_WithdrawOtherToken_AllowsOwnerToWithdrawOtherTokens()
(gas: 4004438)
[PASS]
test_WithdrawOtherToken_RevertsIfTryingToWithdrawDistributionToken()
(gas: 3105127)
[PASS] test_WithdrawTokenFee_AllowsFeeCollectorToWithdrawTokenFees()
(gas: 3243444)
[PASS] test_WithdrawTokenFee_AllowsPartialWithdrawal() (gas:
3258117)
[PASS] test_WithdrawTokenFee_NoFeesToWithdraw() (gas: 3206478)
[PASS] test_WithdrawTokenFee_RevertsForZeroAddressRecipient() (gas:
3225418)
[PASS] test_WithdrawTokenFee_RevertsIfNoFeeCollected() (gas:
3107477)
```

```
[PASS] test_WithdrawTokenFee_RevertsWhenNotTheFeeCollector() (gas: 3230916)
[PASS] test_WithdrawTokenFee_WithAmountGreaterThanReserve() (gas: 3243756)
[PASS] test_Withdraw_AllowsOwnerToWithdrawAfterEndTime() (gas: 3232017)
[PASS] test_Withdraw_RevertsIfNoWithdrawableAmount() (gas: 3333871)
[PASS] test_Withdraw_RevertsIfOwnerTriesDuringClaimPeriod() (gas: 3212856)
Suite result: ok. 97 passed; 0 failed; 0 skipped; finished in 165.75ms (139.19ms CPU time)
```

Ran 43 tests for

test/TokenOpsMerkleDistributorFactory.t.sol:TokenOpsMerkleDistributorFactoryTest

```
[PASS] testNewMerkleDistributorNative() (gas: 2118316)
[PASS] testNewMerkleDistributorNativeWithCustomFee() (gas: 2154990)
[PASS] test_Constructor() (gas: 29249)
[PASS] test_ConstructorParams() (gas: 7286806)
[PASS] test_CreateDistributorWithCustomFee() (gas: 3137178)
[PASS] test_CreateDistributorWithCustomGasFee() (gas: 3117170)
[PASS] test_CreateDistributorWithDefaultGasFee() (gas: 3066237)
[PASS] test_CreateDistributorWithDefaultTokenFee() (gas: 3102469)
[PASS] test_CreateDistributorWithDisabledCustomFee() (gas: 3085813)
[PASS] test_CustomFeeDisableEnableCycle() (gas: 3129965)
[PASS] test_CustomFeeDisabledFallsBackToDefault() (gas: 3109204)
[PASS] test_CustomFeeSwitchPreferredType() (gas: 3155934)
[PASS] test_DefaultFeeTypeDistributionTokenNoCustomFee() (gas: 3102514)
[PASS] test_DisableCustomFee() (gas: 65377)
[PASS] test_EnumValidation_InvalidValueViaLowLevelCall() (gas: 14823)
[PASS] test_NativeDistributorCustomFeeNotGasUsesDefault() (gas: 2178512)
[PASS] test_NewMerkleDistributor() (gas: 3113924)
[PASS] test_NewMerkleDistributorNative_CustomFeeEnabledButNotGas() (gas: 2178617)
[PASS] test_NewMerkleDistributorNative_CustomGasFee() (gas: 2152907)
[PASS] test_NewMerkleDistributorNative_DefaultGasFee() (gas: 2131519)
[PASS] test_ResetGasFee() (gas: 32901)
[PASS] test_ResetTokenFee() (gas: 32937)
[PASS] test_RevertConstructorZeroAddress() (gas: 74776)
[PASS] test_RevertDisableCustomFeeNonOwner() (gas: 73049)
[PASS] test_RevertDisableCustomFeeZeroAddress() (gas: 14300)
[PASS] test_RevertEnableCustomFeeNonOwner() (gas: 16401)
```

```
[PASS] test_RevertEnableCustomFeeZeroAddress() (gas: 15367)
[PASS] test_RevertSetCustomFeeZeroAddress() (gas: 15411)
[PASS] test_RevertSetCustomFee_FeeTooHigh() (gas: 16989)
[PASS] test_RevertSetDefaultGasFeeNotOwner() (gas: 15122)
[PASS] test_RevertSetDefaultTokenFeeNotOwner() (gas: 15078)
[PASS] test_RevertSetDefaultTokenFee_FeeTooHigh() (gas: 14366)
[PASS] test_RevertSetFeeCollectorNonOwner() (gas: 17281)
[PASS] test_RevertSetFeeCollectorNotOwner() (gas: 17311)
[PASS] test_RevertSetFeeCollectorZeroAddress() (gas: 14298)
[PASS] test_RevertSetPreferredFeeTypeZeroAddress() (gas: 15368)
[PASS] test_SetCustomFee() (gas: 97873)
[PASS] test_SetCustomFeeWithInvalidFeeType() (gas: 16080)
[PASS] test_SetDefaultFeeType() (gas: 25471)
[PASS] test_SetDefaultGasFee() (gas: 42863)
[PASS] test_SetDefaultTokenFee() (gas: 42975)
[PASS] test_SetFeeCollector() (gas: 26733)
[PASS]
```

```
test_SetPreferredFeeTypeWithITokenOpsMerkleDistributorFactory()
(gas: 30208)
```

```
Suite result: ok. 43 passed; 0 failed; 0 skipped; finished in
165.52ms (130.23ms CPU time)
```

Ran 44 tests for

test/TokenOpsMerkleDistributorNative.t.sol:TokenOpsMerkleDistributorNativeTest

```
[PASS] test_Claim_AllowsClaimsWithValidMerkleProof() (gas: 2215835)
[PASS] test_Claim_RecordsFirstClaimTimeCorrectly() (gas: 2277289)
[PASS] test_Claim_RevertWhenTransferFails() (gas: 2126592)
[PASS] test_Claim_RevertsWhenClaimNotStarted() (gas: 2103707)
[PASS] test_Claim_RevertsWhenClaimWindowHasEnded() (gas: 2104724)
[PASS] test_Claim_RevertsWhenIndexAlreadyClaimed() (gas: 2221874)
[PASS] test_Claim_RevertsWhenInsufficientNativeTokensInContract()
(gas: 2236950)
[PASS] test_Claim_RevertsWithInsufficientFee() (gas: 2105057)
[PASS] test_Claim_RevertsWithInvalidMerkleProof() (gas: 2109182)
[PASS] test_Claim_TracksNumNativeReservedForFee() (gas: 2275411)
[PASS] test_Claim_WithZeroFee_PayingExtraFee() (gas: 2104910)
[PASS] test_Claim_WithZeroGasFee() (gas: 2187318)
[PASS] test_Constructor_InitializesCorrectly() (gas: 2098860)
[PASS] test_Constructor_RevertsWhenAdminIsZeroAddress() (gas: 48383)
[PASS] test_Constructor_RevertsWhenEndTimeIsInPast() (gas: 48287)
[PASS] test_Constructor_RevertsWhenFeeCollectorIsZeroAddress() (gas:
46391)
[PASS] test_Constructor_RevertsWhenNotEnoughNativeTokens() (gas:
256038)
[PASS] test_Constructor_RevertsWhenStartTimeAfterEndTime() (gas:
```

```
48553)
[PASS]
test_GracePeriodRemainingTime_ReturnsCorrectValueDuringGracePeriod()
(gas: 2212466)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroAfterGracePeriod()
(gas: 2212230)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroBeforeFirstClaim()
(gas: 2080640)
[PASS] test_GracePeriodRemainingTime_ReturnsZeroBeforeStartTime()
(gas: 2080698)
[PASS] test_GracePeriodStatus_ReturnsFalseBeforeFirstClaim() (gas:
2080586)
[PASS] test_GracePeriodStatus_ReturnsFalseWithinGracePeriod() (gas:
2212123)
[PASS] test_GracePeriodStatus_ReturnsTrueAfterGracePeriod() (gas:
2212076)
[PASS] test_UpdateFeeCollector_ChainOfUpdates() (gas: 2314264)
[PASS] test_UpdateFeeCollector_NewFeeCollectorCanWithdrawFees()
(gas: 2303801)
[PASS]
test_UpdateFeeCollector_OldFeeCollectorCannotWithdrawAfterUpdate()
(gas: 2237934)
[PASS] test_UpdateFeeCollector_OnlyCallableByFeeCollector() (gas:
2159117)
[PASS] test_UpdateFeeCollector_RevertsForZeroAddress() (gas:
2080878)
[PASS] test_UpdateFeeCollector_RevertsWhenCalledByNonFeeCollector()
(gas: 2093598)
[PASS] test_WithdrawGasFee_AllowsFeeCollectorToWithdrawGasFees()
(gas: 2234971)
[PASS] test_WithdrawGasFee_AllowsPartialWithdrawal() (gas: 2250131)
[PASS] test_WithdrawGasFee_HandlesFailedTransfers() (gas: 2207072)
[PASS] test_WithdrawGasFee_ResetsNumNativeReservedForFee() (gas:
2233389)
[PASS] test_WithdrawGasFee_RevertsForZeroAddressRecipient() (gas:
2212428)
[PASS] test_WithdrawGasFee_RevertsIfNumNativeReservedForFeeIsZero()
(gas: 2082984)
[PASS] test_WithdrawGasFee_WithAmountGreaterThanReserve() (gas:
2213629)
[PASS] test_WithdrawGasFee_WithZeroFeeCollected_Reverts() (gas:
2187134)
[PASS] test_Withdraw_AllowsOwnerToWithdrawAfterEndTime() (gas:
2255281)
[PASS] test_Withdraw_CalculatesWithdrawableAmountCorrectly() (gas:
2252715)
```

```
[PASS] test_Withdraw_HandlesFailedTransfers() (gas: 2178066)
[PASS] test_Withdraw_RevertsIfNoWithdrawableAmount() (gas: 2345319)
[PASS] test_Withdraw_RevertsIfOwnerTriesDuringClaimPeriod() (gas:
2213355)
Suite result: ok. 44 passed; 0 failed; 0 skipped; finished in
165.49ms (242.05ms CPU time)
```

```
Ran 3 test suites in 261.43ms (496.75ms CPU time): 184 tests passed,
0 failed, 0 skipped (184 total tests)
```

Code Coverage

Code coverage is high for the in-scope contracts. However, the audit team recommends improving branch coverage to over 90% for the `TokenOpsMerkleDistributor` and `TokenOpsMerkleDistributorFactory` contracts.

Fix-Review Update: The client added more tests to improve the code coverage score.

File	% Lines	% Statements	% Branches
contracts/core/BaseTokenOpsMerkleDistributor.sol	98.25% (56/57)	98.55% (68/69)	91.67% (1
contracts/distributors/TokenOpsMerkleDistributor.sol	100.00% (86/86)	100.00% (114/114)	100.00% (30/30)
contracts/distributors/TokenOpsMerkleDistributorNative.sol	100.00% (36/36)	97.92% (47/48)	93.33% (
contracts/factory/TokenOpsMerkleDistributorFactory.sol	100.00% (69/69)	100.00% (64/64)	100.00% (11/11)
test/TokenOpsMerkleDistributor.t.sol	96.15% (25/26)	100.00% (16/16)	100.00%
test/TokenOpsMerkleDistributorFactory.t.sol	75.00% (3/4)	100.00% (2/2)	100.00%
test/TokenOpsMerkleDistributorNative.t.sol	100.00% (2/2)	100.00% (1/1)	100.00%
test/mock/TestERC20.sol	100.00% (2/2)	100.00% (1/1)	100.00%

File	% Lines	% Statements	% Branches
Total	98.94% (279/282)	99.37% (313/315)	97.10% (6)

Changelog

- 2025-05-15 - Initial report
- 2025-05-28 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any

report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party

software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.



© 2026 – Quantstamp, Inc.

TokenOps 3 - Airdrop